

▼ MLOps Assignment 2 - Group 85: Dev Environment Notebook

 **PRIVATE DEV NOTEBOOK - Contains Secrets for GitHub, DagsHub, MLflow, Kaggle, Ngrok, Docker**

This Colab notebook sets up the complete **development environment** for **MLOps_Assignment_2_Group_85** (Cats vs Dogs Classification Pipeline).

 What This Notebook Does (Step-by-Step)

1. **Secret Management**

- Loads Colab secrets: `DAGSHUB_TOKEN`, `GITHUB_TOKEN`, `NGROK_AUTH_TOKEN`, `MLFLOW_TRACKING_URI`, Kaggle API
- Configures Git, DVC, MLflow with secure auth

2. **Repository Setup**

`git clone` → `dvc init` → `dvc remote (DagsHub)` → data versioning

3. **Data Pipeline (M1)**

Kaggle download → `src/data_preprocessing.py` → DVC track (`processed.zip`) → `git push` + `dvc push`

4. **Training Pipeline (M1)**

`python src/train.py` → Logs model to DagsHub MLflow Registry

5. **Live API Deployment and Monitoring Performance (M2, M5)**

FastAPI `app.py` → MLflow model load → Ngrok tunnel → Public `/predict` endpoint

Test: `POST image` → Get cat/dog prediction!

 Prerequisites (Colab Secrets)

Secret Name	Value
<code>DAGSHUB_USERNAME</code>	your-dagshub-username
<code>DAGSHUB_TOKEN</code>	your-dagshub-token
<code>GITHUB_USERNAME</code>	your-github-username
<code>GITHUB_TOKEN</code>	github_pat_xxx
<code>NGROK_AUTH_TOKEN</code>	2abc...
<code>MLFLOW_TRACKING_URI</code>	https://dagshub.com/YOUR/mlops-cats-dogs.mlflow

 Expected Outputs

-  `data/processed.zip.dvc` (tracked)
-  Model in DagsHub MLflow Registry
-  Public Ngrok URL: `https://xxx.ngrok-free.dev/docs`
-  Live `/predict` endpoint (upload animal.jpg → "cat" or "dog")

Run cells top-to-bottom → Full MLOps pipeline live in 5 mins! 

▼ Configure DVC and MLflow

```
!pip install -q dvc dvc-s3 dagshub mlflow -q

from google.colab import userdata
import os

# Load Colab Secrets
DAGSHUB_USER = userdata.get('DAGSHUB_USERNAME')
DAGSHUB_TOKEN = userdata.get('DAGSHUB_TOKEN')
GITHUB_USER = userdata.get('GITHUB_USERNAME')
GITHUB_EMAIL = userdata.get('GITHUB_EMAIL')
GITHUB_TOKEN = userdata.get('GITHUB_TOKEN')

# Set Env Vars for MLflow and DVC
os.environ["MLFLOW_TRACKING_URI"] = userdata.get('MLFLOW_TRACKING_URI')
os.environ["MLFLOW_TRACKING_USERNAME"] = DAGSHUB_USER
os.environ["MLFLOW_TRACKING_PASSWORD"] = DAGSHUB_TOKEN

# Create the hidden kaggle directory
```

```

os.makedirs('/root/.config/kaggle', exist_ok=True)

# Move the file you just uploaded into that directory
!mv kaggle.json /root/.config/kaggle/

# Lock down the file permissions (Kaggle requires this security step)
!chmod 600 /root/.config/kaggle/kaggle.json

print("✅ kaggle.json uploaded, moved, and secured!")

repo_name = "mllops-cats-dogs"
if not os.path.exists(repo_name):
    # Clone repository using GitHub token for authentication
    git_url = f"https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_USER}/{repo_name}.git"
    !git clone $git_url
    print("✅ Repository cloned with Auth!")
else:
    print("ℹ Repository already exists.")

%cd {repo_name}

# Configure Git user identity
!git config --global user.email "{GITHUB_EMAIL}"
!git config --global user.name "{GITHUB_USER}"

if not os.path.exists(".dvc"):
    print("💡 Initializing DVC...")
    !dvc init
    !dvc remote add -d origin https://dagshub.com/{DAGSHUB_USER}/{repo_name}.dvc
    # Configure DVC for DagsHub authentication
    !dvc remote modify origin --local auth basic
    !dvc remote modify origin --local user $DAGSHUB_USER
    !dvc remote modify origin --local password $DAGSHUB_TOKEN
    print("✅ DVC Initialized.")
else:
    # Ensure DVC is configured for DagsHub authentication
    !dvc remote modify origin --local auth basic
    !dvc remote modify origin --local user $DAGSHUB_USER
    !dvc remote modify origin --local password $DAGSHUB_TOKEN
    print("✅ DVC Configured. Pulling data...")
    !dvc pull -r origin

```

[Show hidden output](#)

▼ Data Acquisition & Data Versioning

```

# !pip install -q kaggle
import os
from google.colab import userdata

# 1. Run preprocessing script
!python src/data_preprocessing.py

# . Ensure Git ignores the raw and processed images to prevent freezing
!echo "data/" >> .gitignore

#4. Track the processed data with DVC

#4.1. Zip the processed images into a single archive (the -q flag keeps the output quiet)
!cd data && zip -r -q processed.zip processed/

#4.2. Tell Git to ignore the new zip file so it doesn't freeze Git
!echo "data/processed.zip" >> .gitignore

# 4.3. DVC track the single zip file instead of massive processed files
!dvc add data/processed.zip

# 4.4. Update Git with the new tracking file
!git add data/processed.zip.dvc .gitignore
!git commit --amend -m "M1: Preprocess and DVC for Cats vs Dogs dataset"

# 5. Push! (This will be much faster since it's only 1 file)
!dvc push
!git push origin develop --force

```

```

Collecting          |1.00 [00:00, 129entry/s]
Pushing
!
 0% |                |0/? [00:00<?,    ?files/s]
100% 1/1 [00:00<00:00, 1.86files/s{'info': ''}]
Pushing
Everything is up to date.
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.

```

```
Delta compression using up to 2 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 1.67 KiB | 1.67 MiB/s, done.
Total 14 (delta 5), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (5/5), completed with 4 local objects.
To https://github.com/bashyan/mlops-cats-dogs.git
57770ac..eb58d4ed develop -> develop
```

✓ Run Train from the Repo

```
# 1. Install Dependencies
# !pip install -r requirements.txt -q

# 2. Run Training (This sends logs to DagsHub)
print("🐘 Running Training Pipeline...")
!python src/train.py

print("\n✅ DONE! Code is on GitHub and Model is in DagsHub.")

2026-02-21 12:31:52,332 [INFO] 5292 cuda_device(-1): Unable to register cuda device factory, attempting to register factory for plugin cuda
W0000 00:00:1771677112.309901 5292 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid li
W0000 00:00:1771677112.309933 5292 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid li
W0000 00:00:1771677112.309937 5292 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid li
W0000 00:00:1771677112.309940 5292 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid li
2026-02-21 12:31:52.314547: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2026-02-21 12:31:59,498 [INFO] ✅ Processed dataset directory already exists. Skipping extraction.
2026-02-21 12:31:59,498 [INFO] 🌀 Initializing data generators...
Found 39996 images belonging to 2 classes.
Found 2498 images belonging to 2 classes.
2026/02/21 12:32:00 INFO mlflow.tracking.fluent: Experiment with name 'Cats_vs_Dogs_Prod' does not exist. Creating a new experiment.
2026-02-21 12:32:00,780 [INFO] 🌸 Building baseline CNN architecture...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/input_layer.py:27: UserWarning: Argument `input_shape` is deprecated. Use `
warnings.warn(
2026-02-21 12:32:01.181385: W tensorflow/core/common_runtime/gpu/gpu_bfc_allocator.cc:47] Overriding orig_value setting because the TF_FO
I0000 00:00:1771677121.182882 5292 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13757 MB memor
2026-02-21 12:32:02,530 [INFO] 🚀 Started MLflow run: 897db3b504a94202baf91f03fab19906
2026-02-21 12:32:02,679 [INFO] 🕒 Training started...
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class s
self._warn_if_super_not_called()
Epoch 1/5
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1771677124.004183 5366 service.cc:152] XLA service 0x7847b4028260 initialized for platform CUDA (this does not guarantee t
I0000 00:00:1771677124.004235 5366 service.cc:160] StreamExecutor device (0): Tesla T4, Compute Capability 7.5
2026-02-21 12:32:04.065293: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env v
I0000 00:00:1771677124.297941 5366 cuda_dnn.cc:529] Loaded cudNN version 91002
2026-02-21 12:32:04.791795: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:549] Omitted potentially buggy algor
2026-02-21 12:32:04.927661: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:549] Omitted potentially buggy algor
I0000 00:00:1771677128.399073 5366 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifeti
83/1250 ----- 54s 46ms/step - accuracy: 0.5069 - loss: 1.95002026-02-21 12:32:12.715545: I external/local_xla/xla/servic
2026-02-21 12:32:12.801846: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:549] Omitted potentially buggy algor
1249/1250 ----- 0s 50ms/step - accuracy: 0.5986 - loss: 0.80582026-02-21 12:33:11.694820: I external/local_xla/xla/service
2026-02-21 12:33:11.788536: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:549] Omitted potentially buggy algor
1250/1250 ----- 73s 54ms/step - accuracy: 0.5987 - loss: 0.8055 - val_accuracy: 0.7422 - val_loss: 0.5176
Epoch 2/5
1250/1250 ----- 66s 52ms/step - accuracy: 0.7661 - loss: 0.4907 - val_accuracy: 0.7898 - val_loss: 0.4430
Epoch 3/5
1250/1250 ----- 64s 51ms/step - accuracy: 0.8319 - loss: 0.3818 - val_accuracy: 0.7994 - val_loss: 0.4361
Epoch 4/5
1250/1250 ----- 63s 50ms/step - accuracy: 0.8799 - loss: 0.2830 - val_accuracy: 0.8038 - val_loss: 0.4588
Epoch 5/5
1250/1250 ----- 66s 53ms/step - accuracy: 0.9197 - loss: 0.1969 - val_accuracy: 0.7910 - val_loss: 0.5479
2026-02-21 12:37:43,792 [INFO] 📊 Generating training curves...
2026-02-21 12:37:46,942 [INFO] 📊 Generating confusion matrix...
79/79 ----- 3s 37ms/step
2026-02-21 12:37:50,728 [WARNING] You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This
2026-02-21 12:37:51,864 [INFO] 📁 Model saved locally to models/model.h5
2026/02/21 12:37:51 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/02/21 12:37:56 WARNING mlflow.keras.save: You are saving a Keras model without specifying model signature.
Successfully registered model 'CatsDogs_CNN'.
2026/02/21 12:38:33 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model version to finish creation. Mode
Created version '1' of model 'CatsDogs_CNN'.
2026-02-21 12:38:34,277 [INFO] ✅ Training complete. All artifacts and models successfully logged to MLflow.
🐘 View run abundant-turtle-571 at: https://dagshub.com/bashyan/mlops-cats-dogs.mlflow/#/experiments/0/runs/897db3b504a94202baf91f03fab1
🌟 View experiment at: https://dagshub.com/bashyan/mlops-cats-dogs.mlflow/#/experiments/0

✅ DONE! Code is on GitHub and Model is in DagsHub.
```

```
# Install libraries for dev testing and predictions
!pip install -q pyngrok nest-asyncio
!pip install -q -r requirements.txt
```

Show hidden output

```
import os
import sys
import time
import threading
```

```

import nest_asyncio
import uvicorn
from pyngrok import ngrok
from google.colab import userdata

# --- 0. KILL EXISTING PROCESSES ---
print("🧹 Cleaning up port 8000...")
!fuser -k 8000/tcp
time.sleep(1)

# 1. Setup Environment & Path
# Ensure we are in the project root to find 'app.py' and 'src/'
PROJECT_PATH = '/content/mlops-cats-dogs'
os.chdir(PROJECT_PATH)
if PROJECT_PATH not in sys.path:
    sys.path.append(PROJECT_PATH)

# 2. Import the FastAPI instance
# This must happen AFTER sys.path.append
try:
    # This looks for the 'app' variable inside your 'app.py' file
    from app import app
    print("✅ FastAPI 'app' object imported successfully.")
except ImportError as e:
    print(f"❌ Failed to import 'app' from app.py: {e}")
    print("Make sure your file is named 'app.py' and contains 'app = FastAPI()'")
    raise

# 3. Setup Ngrok
try:
    NGROK_TOKEN = userdata.get('NGROK_AUTH_TOKEN')
except:
    NGROK_TOKEN = "PASTE_YOUR_TOKEN_HERE"

ngrok.set_auth_token(NGROK_TOKEN)
ngrok.kill() # Clean up existing tunnels

# Connect ngrok to the local port
public_url = ngrok.connect(8000)
print(f"\n🌐 Your Public API URL: {public_url.public_url}")
print(f"👉 Swagger UI: {public_url.public_url}/docs\n")

# 4. Configure Async Loop for Uvicorn
nest_asyncio.apply()

# 5. Background Server Function
def run_api():
    try:
        print("🚀 Initializing Uvicorn...")
        # We pass the 'app' object directly.
        # Using 127.0.0.1 is standard for ngrok-to-local tunnels.
        uvicorn.run(
            app,
            host="127.0.0.1",
            port=8000,
            log_level="info",
            access_log=True
        )
    except Exception as e:
        print(f"\n❌ SERVER CRASHED: {e}")
        import traceback
        traceback.print_exc()

# 6. Start the Server Thread
print("🚀 Booting FastAPI server in background...")
server_thread = threading.Thread(target=run_api, daemon=True)
server_thread.start()

# 7. Wait and Verify
print("⌚ Waiting 10 seconds for Model loading (MLFlow)...")
time.sleep(10)

print("\n🌟 Process Complete!")
print("If the model takes a long time to download from DagsHub, the first /predict call might be slow.")

```

🔧 Cleaning up port 8000...

Attempting to load CatsDogs_CNN from MLFlow DagsHub Registry...

Downloading artifacts: 100%

6/6 [00:12<00:00, 2.47s/it]

/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802: UserWarning: Skipping variable loading for optimizer 'rmsprop'
saveable.load_own_variables(weights_store.get(inner_path))

Successfully loaded model from models:/CatsDogs_CNN/None

✅ FastAPI 'app' object imported successfully.

INFO: Started server process [21460]

INFO: Waiting for application startup.

INFO: Application startup complete.

INFO: Uvicorn running on <http://127.0.0.1:8000> (Press CTRL+C to quit)

🌐 Your Public API URL: <https://reagan-nontautomer-ic-unparsimoniously.ngrok-free.dev>

👉 Swagger UI: <https://reagan-nontautomer-ic-unparsimoniously.ngrok-free.dev/docs>

🚀 Booting FastAPI server in background...

🔧 Initializing Uvicorn...

⌚ Waiting 10 seconds for Model loading (MLFlow)...

🌟 Process Complete!

If the model takes a long time to download from DagsHub, the first /predict call might be slow.